NATIONAL ECONOMICS UNIVERSITY
NEU College of Technology



REPORT
STATISTICS

Subject:

PREDICTING CUSTOMER CHURN FOR TELECOMMUNICATIONS PROVIDERS

Group 9: LÊ TRIỆU QUANG MINH
TẠ QUANG HUY
VŨ HOÀNG QUÂN
Class: DS66A

Supervisor: Dr. Thien Van Luong

Ha Noi, October 29th, 2026

INDEX

LIST OF SYMBOLS AND ABBREVIATIONS i

DANH MỤC HÌNH VẼ iv

DANH MỤC BẢNG BIỂU v

1 Business Problem Overview and Dataset Provenance 1

1.1 Executive Overview and Business Context 1

1.2 Domain Context and Dataset Provenance 1

1.3 Raw Dataset Scale and Initial Storage Schema 2

1.4 Summary of Chapter 1 2

2.1 Checking for initial missing values and the "illusion of completeness" . 3

2.2 Remove identifier variables with a high number of unique values (`customerID`) 3

2.3 Data type casting and detecting hidden null values `TotalCharges` . . . 4

2.4 Visual verification and listwise deletion of records with missing values . 4

2.5 Before-and-After Data State Audit 5

2.6 Conclusion to Chapter 2 6

3 DATA OVERVIEW AND CLEANING 7

3.1 Dataset overview 7

3.1.1 Origin and scale 7

3.1.2 Attribute grouping 7

3.1.3 Initial descriptive statistics 7

3.2 Data cleaning 9

3.2.1 Checking for missing values and "hidden" missing values 9

3.2.2 Remove identifier attribute `customerID` 9

3.2.3 Convert types and detect missing values. `TotalCharges` 10

3.2.4 Handling 11 missing records: removing them instead of imputing values . . . 10
3.2.5 Post-cleaning summary . . . 11

4 Feature Engineering and Model Preparation 12
4.1 Exploratory Data Analysis (EDA) . . . 12
4.1.1 Distribution of the target variable . . . 12
4.1.2 Continuous variables: `tenure`, `MonthlyCharges`, `TotalCharges` 13
4.1.3 Categorical variables: `Contract` and `PaymentMethod` . . . 14
4.1.4 Correlation matrix . . . 15
4.1.5 Summary of findings and implications for modeling . . . 16
4.2 Data Preprocessing and Prevention of Data Leakage . . . 16
4.2.1 Feature encoding . . . 16
4.2.2 Train/test split . . . 17
4.2.3 Scaling numerical features . . . 17
4.2.4 Rules for using the test set and remaining limitation . . . 18
4.3 Chapter Summary . . . 18

5 MACHINE LEARNING MODELS AND EXPERIMENTAL DESIGN 19

CHAPTER 5. MACHINE LEARNING MODELS AND EXPERIMENTAL DESIGN 19
5.1 Problem Statement . . . 19
5.2 Model Input Data . . . 19
5.3 Compared Models . . . 20
5.4 Preprocessing Pipeline and Data Leakage Control . . . 21
5.5 Experimental Workflow . . . 22
5.6 Evaluation Metrics . . . 23
5.7 Chapter Summary . . . 24

6 EXPERIMENTAL RESULTS AND EVALUATION 25

CHAPTER 6. EXPERIMENTAL RESULTS AND EVALUATION **25**

6.1 Model Comparison with Default Hyperparameters 25

6.2 Results After Hyperparameter Tuning 26

6.3 Classification Threshold Selection 27

6.4 Evaluation on the Test Set 27

6.5 Practical Implications 29

6.6 Limitations 29

6.7 Chapter Summary 29

7 CONCLUSION AND FUTURE DIRECTIONS **31**

CHAPTER 7. CONCLUSION AND FUTURE DIRECTIONS **31**

7.1 Conclusion 31

7.2 Future Directions 31

MEMBER CONTRIBUTIONS **33**

REFERENCES **33**

LIST OF ABBREVIATIONS AND NOTATIONS

Abbreviation	Full English Term	Description / Meaning
Graduation Thesis	Graduation Thesis / Capstone Project	Undergraduate thesis study
AI	Artificial Intelligence	Machine intelligence field
ML	Machine Learning	Algorithms learning from data
DL	Deep Learning	Deep neural networks
SOTA	State-of-the-Art	Current best performance benchmark
– Architectures & Models		
RNN	Recurrent Neural Network	Sequential neural network
GRU	Gated Recurrent Unit	Gated recurrent mechanism
LSTM	Long Short-Term Memory	Gated memory architecture
CNN	Convolutional Neural Network	Grid/image-based neural network
GNN	Graph Neural Network	Graph-structured neural network
MaRENet	Mask-based Recurrent Expansion Transactional Network	Masked recurrent expansion model
MLP	Multi-Layer Perceptron	Feedforward neural network
– Loss Functions & Optimization		
BCE	Binary Cross-Entropy	Binary classification loss
Focal	Focal Loss	Loss reweighting hard examples
UW	Uncertainty Weighting	Loss weighting via uncertainty
GD	Gradient Descent	First-order optimization method
SGD	Stochastic Gradient Descent	Iterative mini-batch optimization
Adam	Adaptive Moment Estimation	Adaptive learning rate optimizer
– Evaluation & Statistics		

Continued on next page

Abbreviation	Full English Term	Description / Meaning
TP	True Positive	Correct positive predictions
FP	False Positive	Incorrect positive predictions
TN	True Negative	Correct negative predictions
FN	False Negative	Incorrect negative predictions
TPR	True Positive Rate	Sensitivity / Recall metric
FPR	False Positive Rate	Fall-out rate metric
F1	F1-score	Harmonic mean of Precision and Recall
AUC-ROC	Area Under ROC Curve	Receiver operating characteristic area
AUC-PR	Area Under Precision–Recall Curve	Precision-recall curve area
MCC	Matthews Correlation Coefficient	Binary classification quality metric
K-fold CV	K-fold Cross-Validation	Model evaluation resampling method
– Data & Systems		
TE	Transactional Expansion	Contextual transaction enrichment
CNP	Card-Not-Present	Remote payment transactions
CP	Card-Present	Physical terminal transactions
CSV	Comma-Separated Values	Plain text tabular data format
JSON	JavaScript Object Notation	Structured data interchange format
GPU	Graphics Processing Unit	Parallel computation processor
API	Application Programming Interface	Software interaction interface
KPI	Key Performance Indicator	Measurable performance metric
– Mathematical Notations		
$\mathbf{x}_t$	Input feature vector at time step t	Time-indexed feature vector
h_t	Hidden state	Recurrent hidden activation
c_t	Cell state	LSTM memory cell memory
$\sigma(\cdot)$	Sigmoid activation function	Logistic mapping to $(0,1)$

Continued on next page

(Continued: List of Abbreviations and Notations)

Abbreviation	**Full English Term**	**Description / Meaning**
$\tanh(\cdot)$	Hyperbolic tangent function	Non-linear mapping to $(-1, 1)$
$\odot$	Hadamard product	Element-wise multiplication operator

LIST OF FIGURES

Hình 3.1 Matrix of missing values in the original data: no empty cells detected, as missing values are hidden in the form of whitespace. 9

Hình 3.2 Matrix of missing values after type casting `TotalCharges`: 11 missing records revealed. 10

Hình 4.3 Proportion of customers who churned and customers who stayed. . 12

Hình 4.4 Boxplots of `tenure`, `MonthlyCharges` and `TotalCharges` by churn status. 13

Hình 4.5 Distribution (histogram and KDE) of `tenure` and `MonthlyCharges` by churn status. 13

Hình 4.6 Number of customers by contract type and payment method, split by churn status. 14

Hình 4.7 Correlation matrix of the 9 features with the largest absolute correlation with `Churn`, and `Churn` itself. 15

LIST OF TABLES

Bảng 1.2 Initial storage schema of the raw dataset ($N = 7,043$ rows $\times$ 21 columns). *Note: `TotalCharges` is a continuous numerical variable misclassified as `object` due to formatting artifacts. 2

Bảng 2.1 Comparison table of data structures before and after the cleaning process. 6

Bảng 3.2 Grouping 21 attributes of the Telco Customer Churn dataset. . . . 8

Bảng 3.3 TDescriptive statistics for numerical attributes across 7,043 original records. 8

Bảng 3.4 Data state before and after cleaning. 11

Bảng 4.5 Churn rate by contract type and payment method. 14

Bảng 4.6 Summary of EDA findings and their implications for the modeling step. 17

Chapter 1. Business Problem Overview and Dataset Provenance

1.1 Executive Overview and Business Context

In the contemporary telecommunications landscape, market saturation and low switching barriers have fundamentally shifted strategic business priorities from aggressive customer acquisition to proactive customer retention. Customer churn—defined as the phenomenon where a subscriber terminates their relationship or service agreement with a provider—represents one of the most critical threats to recurring revenue stability and long-term corporate valuation. Empirical industry studies consistently demonstrate that acquiring a new telecommunications subscriber is five to seven times more capital-intensive than retaining an existing one, due to substantial marketing overheads, promotional hardware subsidies, and physical infrastructure provisioning costs.

In this report, we present an exhaustive technical and strategic examination of the IBM Telco Customer Churn data. Our primary objectives in this study are structured logically across three chapters:

1. **Chapter 1 (Business Problem & Provenance):** Establish the strategic context of telecommunications churn, introduce the origin and scale of the dataset, and inspect the initial storage schema.

2. **Chapter 2 (Feature Taxonomy & Baseline Statistics):** Classify all 21 attributes into five functional dimensions, document structural dependencies across service variables, and analyze the baseline descriptive statistics of the quantitative features.

3. **Chapter 3 (Data Cleaning & Sanitization Methodology):** Detail the exact programmatic procedures executed to remove non-predictive identifiers, coerce disguised string variables into numeric types, and handle hidden missing values without inducing data leakage.

1.2 Domain Context and Dataset Provenance

The data we examined in our study is the public benchmark *IBM Telco Customer Churn* dataset, retrieved directly from the official IBM Cloud Pak for Data repository (`Telco-Customer-Churn.csv`). It models a fictional yet highly realistic telecommunications provider offering residential landline telephone and broadband internet services to households. Each observational record in the data represents a unique individual customer account, capturing a cross-sectional snapshot at a specific point in time of the subscriber's demographic profile, subscribed service portfolio, contractual obligations, billing preferences, financial expenditure, and current retention status.

1.3 Raw Dataset Scale and Initial Storage Schema

Prior to any data manipulation or cleaning, our raw dataset comprises exactly $7,043$ rows (individual customer records) and 21 columns (distinct attributes), occupying approximately $1.1+$ MB of memory in a standard `pandas` DataFrame structure.

When ingested into Python via `pd.read_csv()`, `pandas` classifies the 21 columns into three underlying storage data types (`dtypes`), as summarized in Table 1.2:

- **Floating-Point Numeric (`float64` — 1 column):** `MonthlyCharges`, which stores decimal currency figures.
- **Integer Numeric (`int64` — 2 columns):** `SeniorCitizen` (encoded as 0 or 1) and `tenure` (encoded as whole months from 0 to 72).
- **String / Categorical Object (`object` — 18 columns):** Includes the account identifier (`customerID`), 15 categorical demographic, service, and billing attributes, the target variable (`Churn`), and one anomalous financial column (`TotalCharges`).

Pandas Data Type	Column Count	Assigned Attributes in Raw Dataset
`float64` (Decimal)	1	`MonthlyCharges`
`int64` (Integer)	2	`SeniorCitizen`, `tenure`
`object` (Text/String)	18	`customerID`, `gender`, `Partner`, `Dependents`, `PhoneService`, `MultipleLines`, `InternetService`, `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`, `StreamingTV`, `StreamingMovies`, `Contract`, `PaperlessBilling`, `PaymentMethod`, `TotalCharges`*, `Churn`

Table 1.2 Initial storage schema of the raw dataset ($N = 7,043$ rows $\times$ 21 columns). *Note: `TotalCharges` is a continuous numerical variable misclassified as `object` due to formatting artifacts.

1.4 Summary of Chapter 1

Chapter 1 has established the commercial imperative of customer retention in the telecommunications sector and introduced the IBM Telco Customer Churn dataset ($7,043$ records across 21 attributes). Initial schema inspection revealed a data type anomaly where numeric fields were misclassified as text strings due to formatting artifacts.

CHAPTER 2. Data Cleaning and Sanitization

Raw enterprise data exported from relational billing systems rarely conforms to the strict mathematical assumptions required by statistical visualization libraries and machine learning algorithms. In our study, we executed a structured, multi-stage data sanitization pipeline on a defensive copy of the DataFrame (`df_cleaned = df.copy()`). Copying the DataFrame first is a critical software engineering best practice that we adopted: it preserves the raw ingested state in memory for auditing and before-and-after shape verification, ensuring idempotency and preventing destructive overwrites.

2.1 Checking for initial missing values and the "illusion of completeness"

In the second stage of our analysis of the data, we executed a standard preliminary null audit using `df.isnull().sum()` alongside a visual missingness matrix generated via the `missingno` library (`msno.matrix(df)`). Surprisingly, `df.isnull().sum()` reported 0 missing values across all 21 columns, and our initial `missingno` matrix rendered as a solid, unbroken dark block across all $7,043$ rows.

However, we recognized that relying solely on a naive `.isnull()` check in `pandas` is a classic data science pitfall. Standard `pandas` CSV parsers (`pd.read_csv()`) only flag cells as `NaN` (Not a Number) if they are completely empty (consecutive commas ,,) or match standard null sentinel strings (such as `'NA'`, `'NULL'`, or `'NaN'`). If a legacy database exports missing numeric fields as whitespace characters (for example, a single space `' '`), `pandas` interprets the whitespace as a valid non-empty string literal. Consequently, the presence of even a single whitespace string forced `pandas` to upcast the entire column's data type from `float64` to generic `object` (string), completely masking the missing values from standard null-detection functions.

2.2 Remove identifier variables with a high number of unique values (`customerID`)

The first explicit transformation we executed in our cleaning pipeline was the removal of the `customerID` column:

- **Execution:** `df_cleaned.drop('customerID', axis=1, inplace=True)`
- **Concrete Reasoning and Mathematical Justification:** In our data of $N = 7,043$ rows, `customerID` has a cardinality of $7,043$ unique values—meaning every single row possesses a distinct identifier with zero repetition. From an information-theoretic perspective, an attribute that is unique for every observation provides zero generalizing power across records. If we retained `customerID` during machine learning preprocessing, it would cause severe structural problems:

1. If One-Hot Encoded, it would explode the feature space by adding $7,043$ sparse binary columns, inducing the Curse of Dimensionality, exhausting system RAM, and rendering matrix inversions singular or computationally intractable.
2. If Label or Target Encoded, tree-based algorithms (such as Random Forests or XGBoost) would exploit arbitrary alphanumeric sort orders to memorize individual training instances, leading to extreme overfitting and a complete failure to generalize to unseen test subscribers.

Therefore, by dropping `customerID`, we immediately reduced dimensionality without losing any behavioral or predictive signal.

2.3 Data type casting and detecting hidden null values `TotalCharges`

During our initial schema inspection (`df.info()`), we identified a glaring anomaly: while `MonthlyCharges` was properly ingested as `float64`, `TotalCharges` was ingested as an `object` (string) column, despite clearly containing decimal currency figures such as `'29.85'`, `'1889.5'`, and `'108.15'`. To rectify this data type mismatch and expose the underlying cause, we executed forced numeric coercion on the data:

- **Execution:** `df_cleaned['TotalCharges'] = pd.to_numeric(df_cleaned['TotalCha errors='coerce')`
- **Concrete Reasoning and Technical Mechanics:** Had we attempted a naive type cast via `.astype(float)` on `TotalCharges`, Python would have immediately raised a fatal `ValueError: could not convert string to float: ' '`. By utilizing `pd.to_numeric()` with the parameter `errors='coerce'`, we instructed the parser to attempt to convert every string token into a 64-bit IEEE 754 floating-point number. Whenever it encountered an unparseable non-numeric string—specifically the blank whitespace strings (`' '`) exported by the billing system—it gracefully caught the exception and substituted the invalid token with a formal `numpy.nan` (Null) marker. Immediately following this operation, we successfully restored the column's dtype to `float64`, enabling vector arithmetic, statistical aggregation, and continuous distribution plotting.

2.4 Visual verification and listwise deletion of records with missing values

Once we applied `pd.to_numeric(..., errors='coerce')`, we audited the newly exposed null values (`hidden_nans = df_cleaned['TotalCharges'].isnull().sum()`) and generated a second `missingno` matrix highlighted in red (`color=(0.8, 0.2, 0.2)`). Our audit revealed exactly 11 hidden missing values in the `TotalCharges` column, visible as distinct horizontal white gaps cutting across the red matrix.

- **Domain Root Cause of the 11 Missing Values:** Why do exactly 11 records in the data lack a `TotalCharges` value? Cross-referencing these 11 rows with our descriptive statistics table (where the minimum value of `tenure` was observed to be 0.0) provided us with the exact domain explanation: all 11 records correspond to brand-new subscribers whose `tenure` is 0 months. Because these customers just signed their service contracts and have not yet completed their first monthly billing cycle, no cumulative invoice has been generated in the accounting ledger, resulting in a blank field for `TotalCharges`.

- **Execution of Listwise Deletion:** `df_cleaned.dropna(inplace=True)`

- **Concrete Reasoning for Dropping vs. Imputing:** When handling missing data, we must carefully choose between listwise deletion (dropping the affected rows) and imputation (filling the missing slots with 0, the mean, the median, or a regression estimate). In our specific analysis, dropping the 11 rows via `dropna()` is rigorously justified by three converging factors:

  1. *Negligible Sample Attrition:* Removing 11 records out of $7,043$ represents a data loss of only 0.156% (retaining 99.844% of the data, or $7,032$ complete rows). In statistical sampling theory, attrition well below the 1% threshold has virtually zero impact on statistical power or variance estimation.
  2. *Avoidance of Distributional Distortion (Mean/Median Imputation Flaw):* If we had blindly applied mean ($\$2,283.30$) or median ($\$1,397.47$) imputation to these 11 rows, we would have created physically impossible, contradictory data records—assigning over a thousand dollars of historical billing to brand-new customers with 0 months of tenure! This would have corrupted the strong linear collinearity between `tenure`, `MonthlyCharges`, and `TotalCharges`.
  3. *Target Variable Integrity for New Sign-ups:* Customers with `tenure == 0` have had zero time to experience service quality, billing cycles, or technical support, and all 11 have `Churn == 'No'`. Removing these 11 unbilled onboarding records ensures that our downstream exploratory analysis and predictive models are trained strictly on active accounts that have experienced at least one full billing cycle (`tenure` ≥ 1).

2.5 Before-and-After Data State Audit

At the conclusion of our data cleaning stage, we ran a comprehensive verification audit confirming the exact structural state of `df_cleaned`, as shown in Table 2.1:

- **Data Shape Transformation:** Reduced from $7,043$ rows $\times$ 21 columns (BEFORE)

to exactly $7,032$ rows $\times$ 20 columns (AFTER)—a net removal of 11 rows and 1 column.

- **Data Type Realignment:** Shifted from `float64(1)`, `int64(2)`, `object(18)` to `float64(2)`, `int64(2)`, `object(16)`. Both `MonthlyCharges` and `TotalCharges` are now properly aligned as `float64`.

- **Final Null Verification:** Our final check (`df_cleaned.isnull().sum()`) confirmed 0 nulls across all 20 remaining columns.

Audit Metric	**Before cleaning (`df`)**	**After cleansing (`df_`**
Number of records(Rows)	7,043	7,032 (reduce by 11 record
Number of attributes (Columns)	21	20 (removed `cust`
Data type distribution (`dtypes`)	`float64(1), int64(2), object(18)`	`float64(2), int64(2)`
Data type of `TotalCharges`	`object` (contains 11 strings `' '`)	`float64` (continuous
Total number of null values (`NaN`)	0 (hide 11 whitespace values)	0 (completely c

Table 2.1 Comparison table of data structures before and after the cleaning process.

2.6 Conclusion to Chapter 2

Chapter 2 has detailed a data-cleaning process with a rigorous mathematical and technical foundation. By creating a secure copy `df_cleaned`, remove the identifier field `customerID`, casting to a floating-point type for `TotalCharges` to disentangle 11 a sequence of hidden whitespace characters and perform deletion based on a list (`dropna()`) available records have `tenure == 0`, We have obtained a standard dataset consisting of 7.032 records and 20 complete set of attributes. This process helps to entirely prevent the risks of dimensionality explosion, overfitting, and distribution distortion—which could otherwise arise from erroneous imputation—thereby establishing a solid foundation for the Exploratory Data Analysis (EDA) phase in Chapter 3..

Chapter 3. DATA OVERVIEW AND CLEANING

This chapter introduces the dataset. *IBM Telco Customer Churn* and a data cleaning process, which includes checking for missing values, removing identifying attributes, and handling the `TotalCharges`. column. The cleaned data is then used for exploratory analysis and modeling in the subsequent chapters.

3.1 Dataset overview

3.1.1 Origin and scale

Research using a dataset *IBM Telco Customer Churn*, a public sample dataset simulating a telecommunications service provider that offers landline telephone and Internet services to households [1]. Each record corresponds to a customer account, capturing—at a specific point in time—demographic information, services currently in use, contracts, payment methods, costs, and churn status.

Before processing, the data includes **7.043 records** and **21 attributes**. According to `pandas`, there is a column of type `float64`, 2 columns `int64` and 18 columns `object`. Within 18 columns `object`, `TotalCharges` column in essence, a continuous variable is being read as a character string; this issue is addressed in Section 3.2.

3.1.2 Attribute grouping

For ease of analysis, the 21 attributes are divided into five functional groups, as shown in the Table. 3.2.

Two structural dependencies need to be noted. First, the value ‘"No phone service"‘ of `MultipleLines` only appears when `PhoneService` equal “No”. Second, six additional Internet service attributes are automatically assigned the value "No internet service" when `InternetService` equal “No”. Therefore, these columns have a structural correlation with each other..

3.1.3 Initial descriptive statistics

Table 3.3 summary of statistics of the numerical columns that `pandas` Identification right from the start (not including) `TotalCharges`).

Some comments: approximately 16.2% elderly customers; `tenure` has a large standard deviation (24.56 months) over a range of values 0–72, shows data that includes both new customers and long-standing, loyal customers; `MonthlyCharges` has a median (70.35) above average (64.76). In other words, many customers fall into a high-tariff bracket, while a smaller group uses only basic voice services, incurring charges of approximately 20 USD. The specific shape of the distribution is examined visually in the

Group	Attribute	Note
Identification	customerID	A unique key for each customer
Demographic	gender, SeniorCitizen, Partner, Dependents	SeniorCitizen encoded as 0/1
Dịch vụ sử dụng	tenure, PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies	tenure the number of months of tenure (0–72); additional services are assigned the value "No internet service" when the customer does not use the Internet
Contracts and costs	Contract, PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges	TotalCharges approximately tenure × MonthlyCharges
Transforming goals	Churn	“Yes” if the customer has left the service within the most recent month

Table 3.2 Grouping 21 attributes of the Telco Customer Churn dataset.

Attribute	Mean	Std	Min	Q1	Median	Q3	Max
SeniorCitizen	0.1621	0.3686	0	0	0	0	1
tenure (tháng)	32.37	24.56	0	9	29	55	72
MonthlyCharges (USD)	64.76	30.09	18.25	35.50	70.35	89.85	118.75

Table 3.3 TDescriptive statistics for numerical attributes across 7,043 original records.

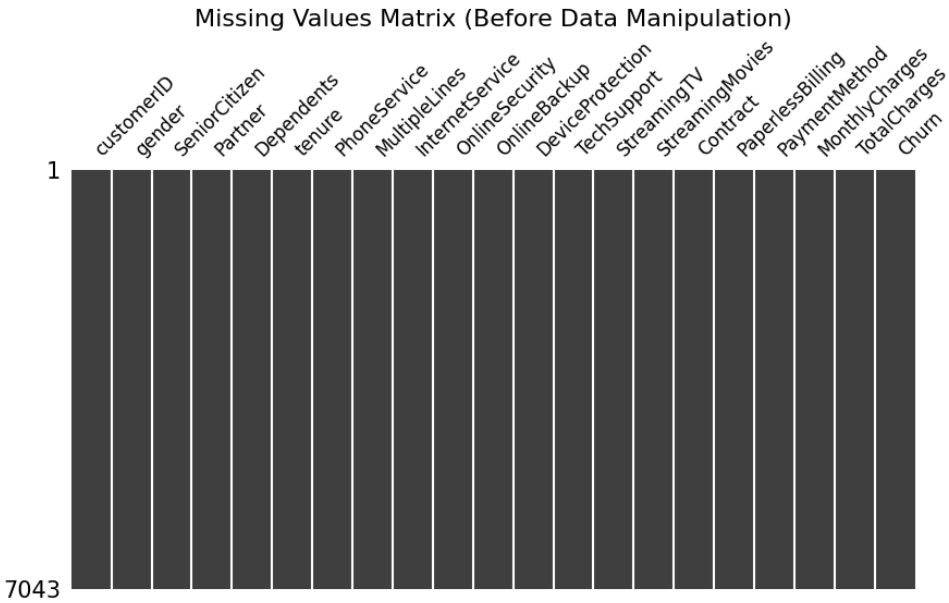
chapter on exploratory data analysis.

3.2 Data cleaning

The entire cleaning process is performed on a copy of the original data.(`df_cleaned = df.copy()`) to retain raw data for the purpose of comparison before and after processing.

3.2.1 Checking for missing values and "hidden" missing values

Command `df.isnull().sum()` and the library's missing data matrix plot `missingno` (Picture 3.1) all indicate that there are no missing values across the 21 columns. However, this result is not reliable: `pd.read_csv()`only consider one cell as `NaN` when that cell is empty or contains non-standard symbols such as 'NA' or 'NaN'. If the source system outputs the missing value as a space character, `pandas` xThat is a valid string, and it also converts the entire column to that type. `object`. This is the reason why `TotalCharges` misread in a certain way.



Picture 3.1 Matrix of missing values in the original data: no empty cells detected, as missing values are hidden in the form of whitespace.

3.2.2 Remove identifier attribute `customerID`

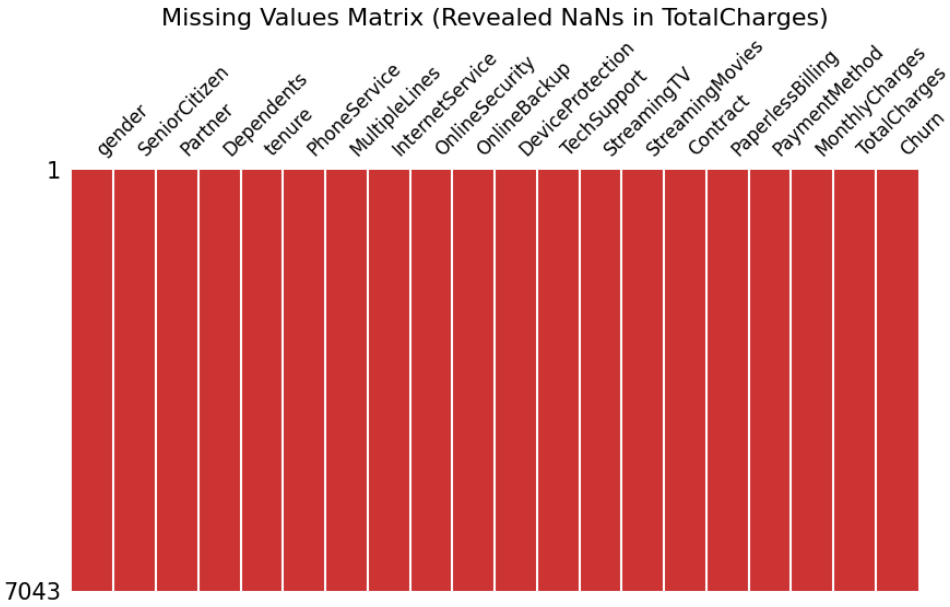
`customerID` column has 7,043 distinct values—one for each record—meaning the column contains no information that can be generalized to new customers. Retaining this column creates two problems: if one-hot encoding is applied, the number of columns increases by thousands, leading to... *curse of dimensionality*; If label encoding is used, tree-based models may "memorize" individual customers, leading to overfitting. Therefore, this column is removed without any loss of predictive information.

3.2.3 *Convert types and detect missing values.* `TotalCharges`

`TotalCharges` column It contains amounts like ‘29.85‘ or ‘1889.5‘ but is stored as a string. Direct type casting is performed using `astype(float)` will report an error `ValueError` due to encountering a sequence of whitespace characters. The group uses:

```
pd.to_numeric(df_cleaned['TotalCharges'], errors='coerce')
```

With parameters `errors='coerce'`, any string that cannot be converted into a number is replaced by `NaN`. After this step, the column has the following style. `float64` and revealed **11 missing values** (Picture 3.2, horizontal white spaces on a red background).



Picture 3.2 Matrix of missing values after type casting `TotalCharges`: 11 missing records revealed.

3.2.4 *Handling 11 missing records: removing them instead of imputing values*

Comparison with descriptive statistics shows that all 11 records have `tenure` equals 0, meaning the customer has just signed up and has not yet gone through their first billing cycle, so there is no total charge. We decided **remove** these records (`dropna()`) instead of entering a value, based on three reasons:

- **The rate of data loss is very small:** 11 out of 7,043 records, equivalent to 0.156
- **Avoid conflicting data:** Imputing the mean (approximately 2,283.30 USD) or the median (approximately 1,397.47 USD) would assign over a thousand USD in cu-

mulative charges to customers with zero months of usage, distorting the relationship between `tenure`, `MonthlyCharges` and `TotalCharges`.

- **Do not affect the label:** All 11 records have `Churn` equal "No",and since these are customers who have not yet experienced the service, their removal does not significantly skew the class distribution..

Removing rows is an operation performed on a record-by-record basis and does not involve learning any statistics from the data; therefore, it can be carried out before splitting the data into training and test sets without causing data leakage.

3.2.5 Post-cleaning summary

Criteria	**Before**	**After**
Dimensions	7,043 rows $\times$ 21 columns	7.032 rows $\times$ 20 columns
Data type	`float64`(1), `int64`(2), `object`(18)	`float64`(2), `int64`(2), `object`(16)
Missing values	0 (obscured by whitespace)	0

Table 3.4 Data state before and after cleaning.

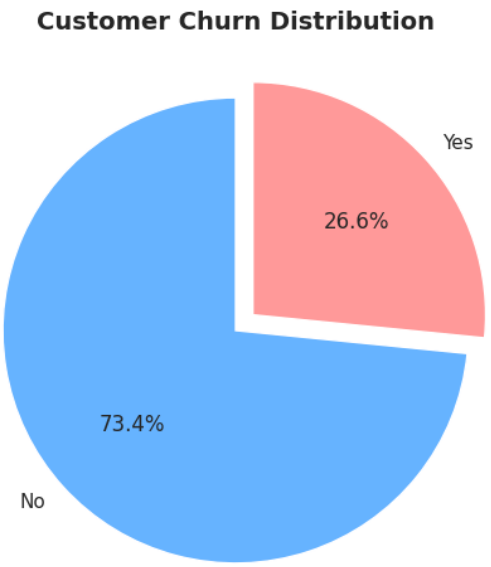
Chapter 4. Feature Engineering and Model Preparation

This chapter has two parts. The first is an exploratory data analysis (EDA) of the *IBM Telco Customer Churn* dataset, cleaned in Chapter 3 (7,032 records and 20 attributes), aimed at identifying the factors associated with customers leaving the service. The second is the data preprocessing that prepares the data for modeling, with particular attention to the order of the steps so that data leakage is avoided, since this is a precondition for trustworthy results in the experimental chapter.

4.1 Exploratory Data Analysis (EDA)

This section analyzes the cleaned data (7,032 records) using `matplotlib` and `seaborn`. The observations below describe *associations* between the features and customer churn; they suggest possible causes but do not establish causality.

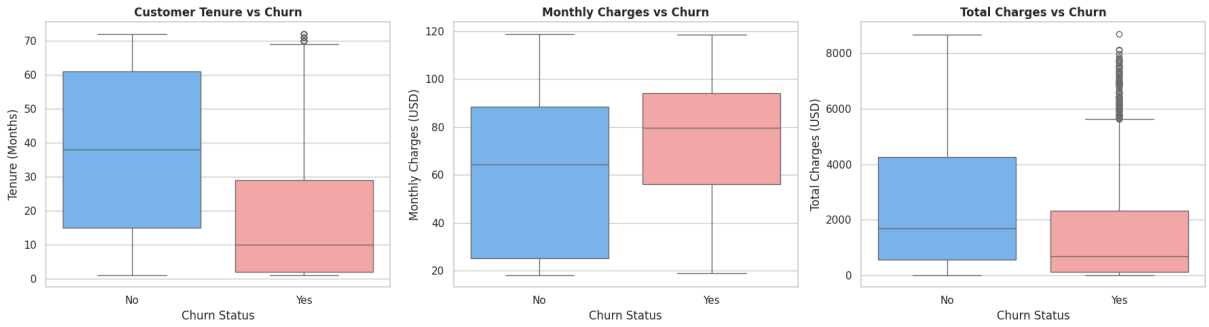
4.1.1 Distribution of the target variable



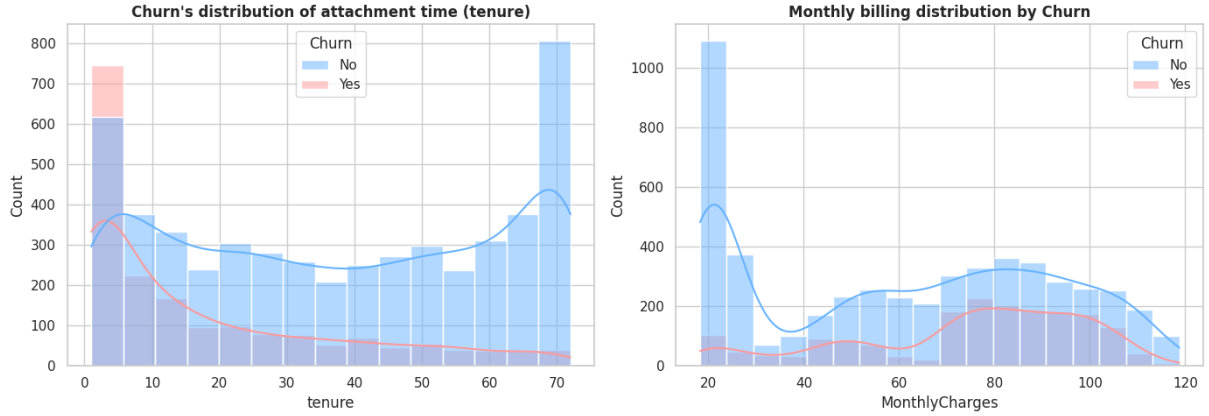
Picture 4.3 Proportion of customers who churned and customers who stayed.

Figure 4.3 shows 5,163 customers who stayed (73.4%) and 1,869 customers who left (26.6%), a ratio of about 2.76 : 1. This is a moderate class imbalance, but large enough that a model always predicting "No" would reach an accuracy of about 73.4% without identifying a single customer about to leave. For this reason, accuracy is not used as the main metric in the experimental chapter; we rely on the F1-score of the churn class together with Recall, Precision and ROC-AUC.

4.1.2 *Continuous variables:* `tenure`, `MonthlyCharges`, `TotalCharges`



Picture 4.4 Boxplots of `tenure`, `MonthlyCharges` and `TotalCharges` by churn status.



Picture 4.5 Distribution (histogram and KDE) of `tenure` and `MonthlyCharges` by churn status.

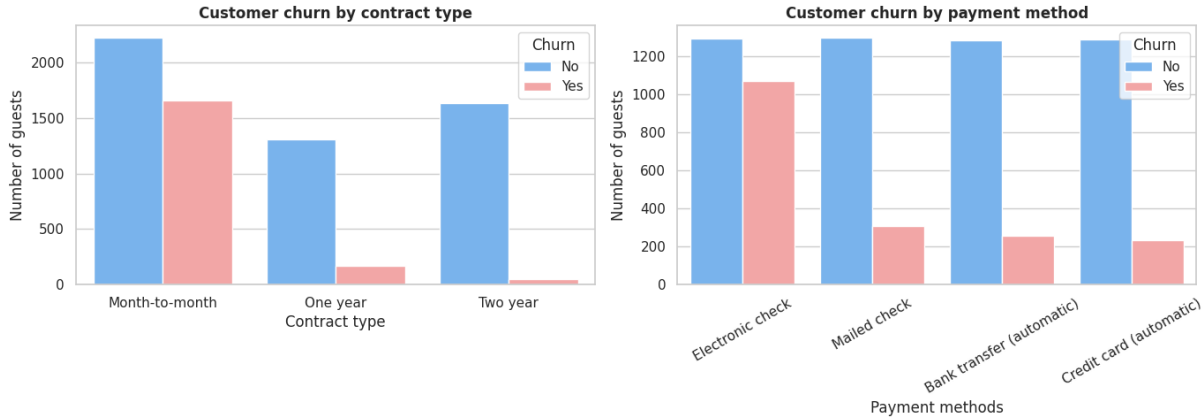
Customer tenure (`tenure`). According to Figures 4.4 and 4.5, customers who stayed have a median tenure of about 38 months, whereas customers who left have a median of only about 10 months. Churners are concentrated in the first months of service, and customers who pass the 12–24 month mark leave noticeably less often. This suggests that the early period carries the highest risk and is the best opportunity for retention efforts.

Monthly charges (`MonthlyCharges`). Churners have a median monthly charge of about \$79.65, higher than customers who stayed (about \$64.50). Among retained customers, the histogram shows a tall peak around \$18–25, corresponding to customers on basic phone-only plans who rarely leave; among churners, the density is concentrated around \$70–105. This indicates a higher risk of leaving in the high-charge group, possibly related to fiber-optic Internet and bundled service packages, but this dataset alone does not allow a causal conclusion.

Total charges (`TotalCharges`). Churners have lower `TotalCharges` (median

about \$700 versus about \$1,680), even though their monthly charges are higher. This is not contradictory: because `TotalCharges` is approximately `tenure` $\times$ `MonthlyCharges`, customers who leave early have not had time to accumulate large total charges. Tenure therefore acts as a confounder when `TotalCharges` is compared directly between the two groups.

4.1.3 Categorical variables: `Contract` and `PaymentMethod`



Picture 4.6 Number of customers by contract type and payment method, split by churn status.

Variable	Value	Total	Stayed	Churned	Churn rate
`Contract`	Month-to-month	3,875	2,220	1,655	42.7%
	One year	1,472	1,306	166	11.3%
	Two year	1,685	1,637	48	2.8%
`PaymentMethod`	Electronic check	2,365	1,294	1,071	45.3%
	Mailed check	1,604	1,296	308	19.2%
	Bank transfer (automatic)	1,542	1,284	258	16.7%
	Credit card (automatic)	1,521	1,289	232	15.3%

Table 4.5 Churn rate by contract type and payment method.

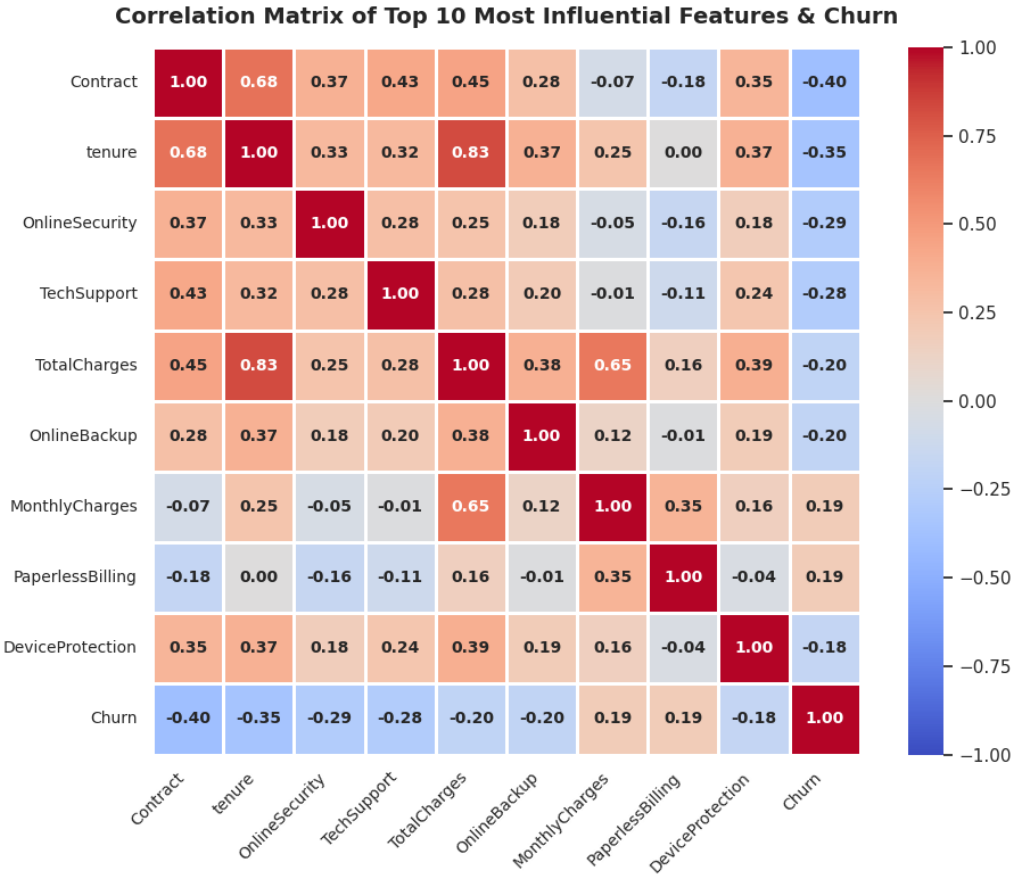
Contract type. Figure 4.6 and Table 4.5 show a very large gap: customers on month-to-month contracts churn at 42.7%, one-year contracts at 11.3%, and two-year contracts at only 2.8%. Short-term contracts carry no exit commitment, so customers can switch providers more easily; this is one of the most discriminative features in the dataset.

Payment method. Customers paying automatically (bank transfer, credit card) have low churn rates of about 15–17%. Customers paying by electronic check churn at 45.3% and account for about 57% of all churners (1,071 out of 1,869). One possible

explanation is that manual payment forces customers to review their bill every month, but this is only a hypothesis requiring further verification; this group may also differ in age, plan type or tenure.

4.1.4 Correlation matrix

To get an overview of how strongly each feature is related to `Churn`, we computed the Pearson correlation matrix on the whole cleaned dataset, temporarily encoding categorical variables as category codes, and then selected the 9 features with the largest $|r|$ with `Churn`. The result is shown in Figure 4.7.



Picture 4.7 Correlation matrix of the 9 features with the largest absolute correlation with `Churn`, and `Churn` itself.

Main observations:

- `Contract` and `tenure` have the strongest negative correlations with `Churn` (-0.40 and -0.35 respectively), consistent with Sections 4.1.2 and 4.1.3: the longer the contract and the longer the tenure, the lower the likelihood of leaving.

- The four add-on services `OnlineSecurity` (-0.29), `TechSupport` (-0.28), `OnlineBackup` (-0.20) and `DeviceProtection` (-0.18) are all negatively correlated with `Churn`:

customers who use these services tend to leave less often. These services are also positively correlated with `Contract` and `tenure` (0.18–0.43), so part of the association may come from customers with more services also staying longer.

- `MonthlyCharges` and `PaperlessBilling` are both weakly positively correlated with `Churn` (0.19), in line with the observation that churners pay higher monthly charges.

- `TotalCharges` is very strongly correlated with `tenure` (0.83) and fairly strongly with `MonthlyCharges` (0.65), as expected from the approximate relation `TotalCharges` $\approx$ `tenure` $\times$ `MonthlyCharges`. `Contract` and `tenure` are also notably correlated (0.68). This multicollinearity should be kept in mind when interpreting linear models such as Logistic Regression.

- No feature has a large $|r|$ with `Churn`: the maximum is only 0.40 (`Contract`). Churn therefore cannot be predicted from a single feature, and a model that combines many features is needed.

Limitations of this matrix. The Pearson coefficient only measures linear association, and assigning numeric codes to nominal categorical variables creates an artificial ordering (for example, "No internet service" receives a code between "No" and "Yes"), so the coefficients of such variables are indicative only. One illustration is that `PaymentMethod` does not appear among the top 9 features, even though Section 4.1.3 shows that *Electronic check* users churn at an unusually high rate; this is most likely because the codes follow alphabetical order, so the relationship is no longer monotonic. The matrix is therefore used for exploration only: we do **not** use it to select features for the models and keep all features, so this step does not cause data leakage even though it is computed on the whole dataset.

4.1.5 Summary of findings and implications for modeling

4.2 Data Preprocessing and Prevention of Data Leakage

4.2.1 Feature encoding

The target variable `Churn` is mapped as "Yes" $\rightarrow$ 1 and "No" $\rightarrow$ 0. The remaining categorical variables are one-hot encoded with `pd.get_dummies(drop_first=True)`; dropping one reference level per variable avoids perfect multicollinearity among the binary columns. After this step the dataset has **7,032 rows** $\times$ **31 columns** (30 features and the target) **[verify against the line** `Dataset shape ready for split` **in the notebook]**.

One-hot encoding is a deterministic transformation of each record's own values;

Finding	Evidence	Implication for modeling
Moderate class imbalance	26.6% churn, ratio 2.76 : 1	Stratified splitting; evaluate with F1, Recall and ROC-AUC rather than accuracy
New customers leave more often	Median tenure: 10 months (churned) vs. 38 months (stayed)	Tenure (`tenure`) is an important feature
Short-term contracts are high risk	Churn 42.7% vs. 2.8% (Two-year)	Contract (`Contract`) must be fully encoded
Manual payment is high risk	Electronic check: 45.3%	Keep `PaymentMethod` as one-hot columns
Multicollinearity among charge variables	`TotalCharges` $\approx$ `tenure` $\times$ `MonthlyCharges`	Take care when interpreting Logistic Regression

Table 4.6 Summary of EDA findings and their implications for the modeling step.

it estimates no parameters from the data or the labels, so performing it before the split does not expose any test-set information. All levels of the categorical variables occur in large numbers, so there is no risk of a rare level appearing only in the test set.

4.2.2 Train/test split

The data is split 80/20 using `train_test_split` with `stratify=y` and `random_state=42`. Stratification keeps the churn rate at about 26.6% in both sets: roughly **5,625** training records and **1,407** test records **[verify against the notebook]**. The test set is held out until model selection and hyperparameter tuning are finished.

4.2.3 Scaling numerical features

The three continuous variables (`tenure`, `MonthlyCharges`, `TotalCharges`) are standardized with `StandardScaler` (Z-score). The most important point is the **order of operations**:

1. Split the data into training and test sets first.
2. `fit` the scaler (computing the mean and standard deviation) **on the training set only**.
3. Use those same parameters to `transform` the test set.

If the mean and standard deviation were computed on the whole dataset before splitting, information from the test set would "leak" into training and make the evaluation more optimistic than reality. Binary columns produced by one-hot encoding do not need scaling.

4.2.4 Rules for using the test set and remaining limitation

To keep the evaluation objective, we apply the following rules:

- The six models are compared using stratified 5-fold cross-validation (`StratifiedKFold`) on the training set only.
- Hyperparameter tuning (`GridSearchCV`) is performed on the training set only.
- The test set is used exactly once to report the final result; the classification threshold is never adjusted using the test set.

Limitation. The scaler is fitted on the whole training set before cross-validation, so in each fold the validation portion has indirectly contributed to the mean and standard deviation. This may make the cross-validation scores slightly higher than the true performance, mainly for Logistic Regression and KNN (tree-based models are unaffected because they do not depend on feature scale). The test-set scores are not affected. The standard remedy is to place the scaler and the model in a single `Pipeline` so that the scaler is refitted inside every fold; we note this as a direction for improvement in the conclusion.

4.3 Chapter Summary

This chapter explored the factors associated with churn (tenure, contract type, payment method, monthly charges) and set up a preprocessing procedure free of data leakage. The resulting data is used in the experimental chapter to compare six classification models and to evaluate them on the test set.

Chapter 5. MACHINE LEARNING MODELS AND EXPERIMENTAL DESIGN

Chapter 4 brought the data into a clean state, outlined findings from EDA, and established data-leakage-free preprocessing principles. This chapter moves on to machine learning: formulating the churn prediction task, describing model inputs, comparing algorithms, outlining the experimental workflow, and defining evaluation metrics. All setups below strictly follow the group's notebook code so that the results in Chapter 6 can be reproduced (`random_state = 42`).

5.1 Problem Statement

Churn prediction is formulated as a supervised binary classification task. Given a feature vector x for a customer, the model estimates the probability $P(\text{Churn} = 1 \mid x)$; the predicted label is "departed" if this probability is greater than or equal to a threshold t, and "stayed" otherwise. The positive class represents departed customers (Churn $= 1$), accounting to 26.6% of the data (1,869/7,032), with a class ratio of approximately 2.76 : 1 (Section 4.1.1).

Two characteristics of the problem guide the entire experimental design:

- **Asymmetric Error Costs.** As stated in Section 1.1, acquiring a new subscriber costs about five to seven times more than retaining an existing one. Missing a customer about to churn (false negative) is therefore typically costlier than sending a retention offer to a customer who would have stayed anyway (false positive). This makes the Recall of the churn class crucial; however, if Recall is maximized alone, the model would label nearly every customer as "departed". The group therefore chooses the F1-score of the churn class as the model selection metric, accompanied by Precision, Recall, ROC-AUC, and PR-AUC.

- **Moderate Class Imbalance.** A model that always predicts "stayed" achieves an accuracy of about 73.4% but fails to detect any departing customers, so accuracy is not used as the primary metric.

5.2 Model Input Data

In addition to the steps in Section 4.2.1, before one-hot encoding, the group groups the values "No internet service" (across six supplementary internet service columns: OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies) and "No phone service" (of MultipleLines) into "No". The reason is that information regarding "customers without internet" or "without phone" is fully repre-

sented by InternetService and PhoneService; keeping a separate level in each individual column merely creates binary columns with structural redundancies (two dependency relationships noted in Section 3.1.2) without adding new information.

The target variable is then mapped "Yes" $\rightarrow$ 1, "No" $\rightarrow$ 0, and the remaining categorical variables are one-hot encoded using `pd.get_dummies(drop_first=True)`. The resulting dataset contains 7,032 rows $\times$ 24 columns, comprising **23 features** and the Churn target variable.

Table 5.1 Structure of the 23 input features.

Group	Features	Column Count
Continuous (normalized)	tenure, MonthlyCharges, TotalCharges	3
Demographics	SeniorCitizen, gender_Male, Partner_Yes, Dependents_Yes	4
Phone / Internet Services	PhoneService_Yes, MultipleLines_Yes, InternetService_Fiber optic, InternetService_No	4
Supplementary Services	OnlineSecurity_Yes, OnlineBackup_Yes, DeviceProtection_Yes, TechSupport_Yes, StreamingTV_Yes, StreamingMovies_Yes	6
Contract and Payment	Contract_One year, Contract_Two year, PaperlessBilling_Yes, PaymentMethod_Credit card (automatic), PaymentMethod_Electronic check, PaymentMethod_Mailed check	6
Total		**23**

The dropped reference level for each variable is the first level in alphabetical order: InternetService = DSL, Contract = Month-to-month, PaymentMethod = Bank transfer (automatic). This transformation is determined per record without estimating parameters from data, so performing it prior to data splitting causes no data leakage (see Section 4.2.1).

Data is split 80/20 with label stratification (`stratify = y`, `random_state = 42`), yielding **5,625 training records** and **1,407 test records**, where the test set contains 374 departing customers and 1,033 staying customers (the churn rate remains approximately 26.6%).

5.3 Compared Models

The group compares six classification algorithms representing different model families, along with a baseline model (Dummy) as a reference point.

Table 5.2 Models and default configurations before tuning.

Model	Algorithm Family	Default Configuration
Dummy Baseline	Baseline model	`strategy = "stratified"`
Logistic Regression (LR)	Linear	`max_iter = 2000`
Decision Tree (DT)	Single decision tree	Scikit-learn default
Random Forest (RF)	Bagging ensemble	`n_estimators = 120`
KNN	Distance-based	`n_neighbors = 11`
AdaBoost	Boosting ensemble	`n_estimators = 100`
Gradient Boosting (GB)	Boosting ensemble	`n_estimators = 100`

Dummy Baseline generates random predictions matching the training set class distribution without using any features. This model indicates the score level "by chance" (e.g., PR-AUC approximately equals the positive class proportion of 0.266); all actual models must outperform this benchmark.

Logistic Regression models the log-odds of churn as a linear function of features: $P(\text{Churn} = 1 \mid x) = 1/(1+\exp(-(w \cdot x + b)))$. Its advantages are simplicity, speed, and interpretability via coefficients; its limitation is that it only captures linear decision boundaries and is sensitive to multicollinearity among tenure, MonthlyCharges, and TotalCharges (Section 4.1.4).

Decision Tree partitions the feature space using sequential threshold conditions. The tree is highly interpretable, but without depth restrictions, it overfits the training data and exhibits high variance.

Random Forest trains multiple trees on bootstrap samples with random feature subsets and averages their probabilities. Averaging reduces the variance of individual trees and captures non-linear interactions, such as those among contract type, tenure, and internet type.

KNN predicts based on the labels of the k nearest neighbors. Because it relies directly on distances, KNN (along with LR) requires normalized numerical features.

AdaBoost and Gradient Boosting sequentially build multiple weak learners (shallow trees), with each new learner focusing on correcting the errors of previous ones by increasing weights of misclassified samples (AdaBoost) or fitting the gradients of the loss function (Gradient Boosting). These two models are typically strong on tabular data but lack a `class_weight` parameter, which affects results in Section 6.2.

5.4 Preprocessing Pipeline and Data Leakage Control

Three continuous variables (tenure, MonthlyCharges, TotalCharges) are Z-score normalized using StandardScaler; remaining binary columns are passed through (`remainder`

= "passthrough"). The normalization is wrapped in a ColumnTransformer and combined with each model into a single Pipeline (prep → model).

This design has an important consequence: when using the Pipeline in cross-validation and GridSearchCV, the normalizer is refit exclusively on the training split of each iteration, and the validation split of that iteration does not contribute to its mean or standard deviation. Consequently, the indirect leakage issue mentioned in Section 4.2.4 (where the scaler is fit before cross-validation) **no longer exists** in the final setup. Normalization affects only LR and KNN; tree-based models are scale-invariant.

5.5 Experimental Workflow

The workflow consists of four fixed-order steps; the test set appears only in the final step.

1. **Model comparison with default hyperparameters.** Using 5-fold stratified cross-validation (StratifiedKFold, shuffle = True, random_state = 42) on the training set, reporting mean Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC, and the standard deviation of F1 across folds.

2. **Hyperparameter tuning.** For each model, GridSearchCV (with the same 5-fold splitter, scoring = F1, refit = True) searches for the best configuration on the training set. The model with the highest cross-validation F1 is selected as the final model.

3. **Classification threshold selection.** For the chosen model, out-of-fold probabilities are collected on the training set via cross_val_predict, scanning threshold t from 0.20 to 0.79 (step 0.01) to select t maximizing F1. The threshold is determined entirely on the training data.

4. **Final evaluation.** The final model is evaluated once on the 1,407-record test set using the threshold selected in step 3 (while also reporting the default 0.50 threshold for comparison).

Table 5.3 GridSearchCV hyperparameter grid.

Model	Hyperparameters and Tested Values	Combinations
Logistic Regression	`C` $\in \{0.1, 0.5, 1, 2, 5\}$; `class_weight` $\in$ {None, balanced}	10
Decision Tree	`max_depth` $\in \{3, 5, 8,$ None$\}$; `min_samples_leaf` $\in \{5, 10, 20\}$; `class_weight` $\in$ {None, balanced}	24
Random Forest	`n_estimators` $\in \{100, 200\}$; `max_depth` $\in$ {None, 10, 20}; `min_samples_leaf` $\in \{2, 5\}$; `class_weight` $\in$ {None, balanced}	24
KNN	`n_neighbors` $\in \{5, 11, 21, 31\}$; `weights` $\in$ {uniform, distance}; `p` $\in \{1, 2\}$	16
AdaBoost	`n_estimators` $\in \{50, 100, 150\}$; `learning_rate` $\in \{0.1, 0.5, 1.0\}$	9
Gradient Boosting	`n_estimators` $\in \{80, 140, 200\}$; `learning_rate` $\in \{0.03, 0.08, 0.15\}$; `max_depth` $\in \{1, 2, 3\}$	27

The parameter `class_weight = "balanced"` assigns higher weight to the minority class (churn) in the loss function. This handles class imbalance at the algorithmic level without requiring synthetic sample generation, thus leaving data unchanged and avoiding resampling-related leakage risks.

5.6 Evaluation Metrics

Let TP, FP, FN, and TN represent true positives, false positives, false negatives, and true negatives respectively (positive class is churn).

- **Precision** = $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$: proportion of customers flagged as departing who actually departed.
- **Recall** = $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$: proportion of actual departing customers successfully detected by the model.
- **F1** = $2 \cdot \mathrm{Precision} \cdot \mathrm{Recall}/(\mathrm{Precision}+\mathrm{Recall})$: harmonic mean of precision and recall, used as the model selection metric.
- **ROC-AUC**: probability that the model ranks a departing customer higher than a staying customer, threshold-independent; random baseline is 0.5.
- **PR-AUC** (average precision): area under the Precision–Recall curve, more sensitive to minority classes than ROC-AUC; random baseline equals the positive class ratio (approx. 0.266).

- **Accuracy** is reported for reference only as it does not reflect the ability to detect departing customers.

5.7 Chapter Summary

Chapter 5 formulated the churn prediction problem as binary classification with asymmetric error costs, described 23 input features and the 5,625/1.407 split, introduced six algorithms along with a baseline model, and outlined the four-step workflow (default comparison, tuning, threshold selection, single test evaluation) within a Pipeline to prevent data leakage. Results are presented in Chapter 6.

Chapter 6. EXPERIMENTAL RESULTS AND EVALUATION

6.1 Model Comparison with Default Hyperparameters

Table 6.1 5-fold cross-validation results on the training set (averaged across folds, default hyperparameters, 0.50 threshold), sorted by F1.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	F1 Std
Logistic Regression	0.803	0.655	0.546	**0.595**	0.846	0.660	0.023
AdaBoost	0.804	0.670	0.521	0.586	0.849	0.666	0.010
Gradient Boosting	0.801	0.662	0.511	0.576	0.848	0.664	0.016
KNN	0.782	0.597	0.553	0.574	0.818	0.572	0.025
Random Forest	0.793	0.645	0.496	0.561	0.827	0.631	0.015
Decision Tree	0.733	0.497	0.514	0.505	0.663	0.385	0.019
Dummy Baseline	0.608	0.265	0.268	0.266	0.499	0.266	0.020

Key observations:

- **All models outperform the baseline.** Dummy achieved an ROC-AUC of 0.499 (exact random level) and PR-AUC of 0.266 (exact positive class ratio), confirming no obvious evaluation bugs and that the data genuinely contains predictive signals.

- **The top four models perform comparably in ranking ability.** LR, AdaBoost, and Gradient Boosting achieved ROC-AUCs between 0.846–0.849 and PR-AUCs of 0.660–0.666; differences are smaller than fold-to-fold F1 standard deviations (0.010–0.025). LR ranking first in F1 at this stage is insufficient to declare it superior.

- **Recall at the 0.50 threshold is low.** Most models achieved a Recall of only 0.50–0.55, missing about half of departing customers despite 0.80 accuracy. This is a typical symptom of models leaning toward the majority class under class imbalance, motivating the use of `class_weight` and threshold tuning.

- **Unconstrained Decision Tree overfits.** ROC-AUC was only 0.663 and PR-AUC 0.385, significantly lower than ensemble models, consistent with single trees memorizing training data.

- **AdaBoost achieved the highest ROC-AUC and PR-AUC but ranked second in F1**, due to high Precision (0.670) paired with low Recall (0.521) at threshold

0.50. This indicates fixed-threshold F1 and threshold-independent metrics can rank models differently.

6.2 Results After Hyperparameter Tuning

Table 6.2 Best cross-validation F1 after GridSearchCV and corresponding configurations (threshold 0.50).

Model	Default F1	Tuned F1	Gain	F1 Std	Best Configuration
Random Forest	0.561	**0.636**	+0.075	0.006	`class_weight = balanced; max_depth = 10; min_samples_leaf = 5; n_estimators = 100`
Logistic Regression	0.595	0.634	+0.039	0.008	`C = 0.5; class_weight = balanced`
Decision Tree	0.505	0.607	+0.102	0.009	`class_weight = balanced; max_depth = 5; min_samples_leaf = 10`
KNN	0.574	0.598	+0.024	0.015	`n_neighbors = 31; p = 1; weights = uniform`
Gradient Boosting	0.576	0.586	+0.010	0.022	`learning_rate = 0.08; max_depth = 3; n_estimators = 200`
AdaBoost	0.586	0.586	+0.000	0.010	`learning_rate = 1.0; n_estimators = 100`

Key observations:

- `class_weight = "balanced"` **is the primary driver of performance gains.** All three models supporting this parameter (Random Forest, Logistic Regression, Decision Tree) selected "balanced", achieving the largest F1 gains (+0.075, +0.039, +0.102). Conversely, AdaBoost and Gradient Boosting lacked this parameter and showed virtually no improvement (+0.000 and +0.010). Thus, most tuning benefits stemmed from handling class imbalance rather than increasing model complexity.

- **Random Forest was selected, but its margin over Logistic Regression is negligible.** RF achieved an F1 of 0.636 versus 0.634 for LR; the 0.002 gap is smaller than cross-validation standard deviations (0.006–0.008), meaning the two models are statistically indistinguishable. Following preset rules (highest cross-validation F1), Random Forest was chosen; if interpretability is prioritized, Logistic Regression serves as a sound alternative with equivalent performance.

- **GridSearchCV peak scores exhibit slight optimistic bias**, being selected as maximum values across multiple configurations on the same data. Final figures therefore require confirmation on a held-out test set (Section 6.4).

6.3 Classification Threshold Selection

Based on out-of-fold training probabilities, the threshold maximizing F1 was $t = 0.46$, with F1 $= 0.639$. This value is merely 0.003 higher than the cross-validation F1 at threshold 0.50 (0.636; noting differing calculation methods between fold averaging and pooled out-of-fold predictions), indicating that the F1-threshold curve is relatively flat around 0.50. A slightly lower threshold makes the model more inclined to label instances as "departed", increasing Recall while slightly lowering Precision.

6.4 Evaluation on the Test Set

Table 6.3 Results of the tuned Random Forest on the test set (1,407 records).

Threshold	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
0.50 (default)	0.758	0.531	0.767	0.628	0.838	0.639
0.46 (selected)	0.750	0.519	0.807	**0.632**	0.838	0.639

Table 6.4 Confusion matrix at threshold 0.46 (rows: actual labels; columns: predicted labels).

	Predicted: Stayed	Predicted: Departed	Total
Actual: Stayed	TN = 753	FP = 280	1,033
Actual: Departed	FN = 72	TP = 302	374

Table 6.5 Classification report at threshold 0.46.

Class	Precision	Recall	F1	Sample Count
Stayed	0.913	0.729	0.811	1,033
Departed	0.519	0.807	0.632	374
Macro average	0.716	0.768	0.721	1,407
Weighted average	0.808	0.750	0.763	1,407

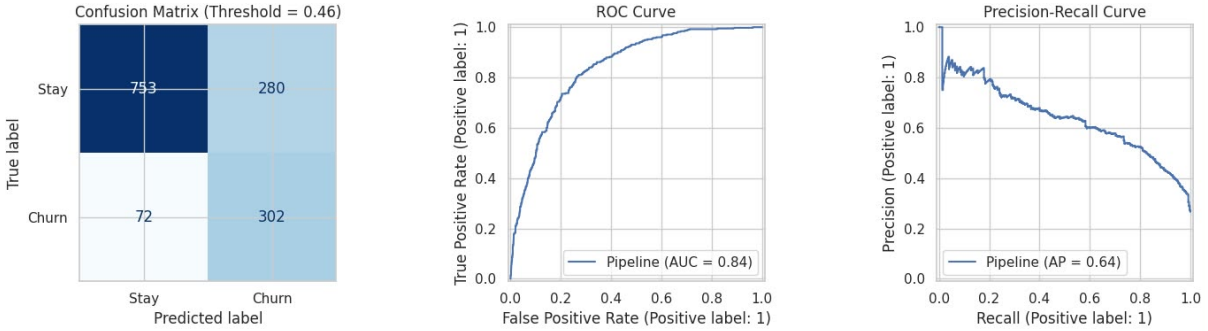


Figure 6.1 Confusion matrix (threshold 0.46), ROC curve (AUC = 0.84) and Precision–Recall curve (AP = 0.64) of the tuned Random Forest on the test set (1,407 records).

Key observations:

- **Test performance is consistent with cross-validation.** The test set F1 for the churn class was 0.632, closely matching the 0.636–0.639 range on the training set, indicating no overfitting caused by hyperparameter tuning or threshold selection.

- **Strong churn detection capability.** The model detected 302 out of 374 actual departing customers (Recall = 0.807), missing only 72. ROC-AUC = 0.838 and PR-AUC = 0.639, approximately 2.4 times the random baseline of 0.266.

- **The trade-off is increased false alarms.** Precision = 0.519 implies that only about half of customers flagged as "departed" (302/582) actually departed; 280 staying customers were falsely flagged. The model trades precision for recall, aligning with the asymmetric error costs in Section 5.1.

- **Reduced Accuracy compared to default models (0.750 vs. approx. 0.80)** is a natural consequence of prioritizing minority class Recall rather than a sign of inferiority; this is why accuracy was excluded as the primary metric (Section 4.1.1).

- **The effect of threshold selection is modest.** Lowering the threshold from 0.50 to 0.46 increased Recall by 0.040 (from 0.767 to 0.807), decreased Precision by 0.012, and raised F1 by only 0.004. Most model quality stems from `class_weight` rather than thresholding. Comparing both thresholds on the test set is descriptive only, as the 0.46 threshold was decided beforehand on training data.

6.5 Practical Implications

The model can be deployed to rank customers by departure probability and target high-risk groups for retention offers. On the test set, out of 582 flagged customers, 302 were genuine churn risks—nearly double the random baseline rate of 26.6

Retention campaigns are profitable only when expected value from retained customers exceeds incentive costs: if v is remaining customer value, q retention probability, and c incentive cost, offering incentives to a group with Precision p is worthwhile when $p \cdot q \cdot v > c$. Companies must estimate c, q, and v from real business data to adjust thresholds based on costs rather than F1. EDA findings in Chapter 4 suggest intervention priorities: newer customers (median tenure around 10 months for churners), month-to-month contracts (42.7

6.6 Limitations

- **Single train/test split and small test set.** With 374 positive samples, metric uncertainty is substantial. Approximate 95

- **Top models are statistically tied.** Random Forest and Logistic Regression differed by only 0.002 in cross-validation F1, well within standard deviation bounds.

- **Small hyperparameter grid.** Only 9 to 27 combinations were tested per model, and boosting models were not evaluated with class weights or stronger variants like XGBoost and LightGBM.

- **Synthetic and cross-sectional data.** The IBM Telco dataset is simulated and cross-sectional, preventing chronological evaluation (training on past data, testing on future) and leaving real-world generalizability unverified.

- **Lack of probability calibration and explainability.** The notebook did not evaluate predicted probability calibration or extract feature importances, leaving key drivers unidentified.

- **F1-optimal threshold assumes equal precision and recall importance**, which may not reflect real cost structures (Section 6.5).

6.7 Chapter Summary

On the training set, all models outperformed the baseline, ensemble models and Logistic Regression showed comparable ranking ability (ROC-AUC around 0.85), while Recall at the 0.50 threshold was only around 0.5. Tuning with `class_weight = "balanced"` was the most impactful factor for improving F1, bringing Random Forest (cross-validation

F1 of 0.636) to the top, closely followed by Logistic Regression (0.634). At a threshold of 0.46, Random Forest achieved a test set Recall of 0.807, Precision of 0.519, F1 of 0.632, ROC-AUC of 0.838, and PR-AUC of 0.639, consistent with cross-validation results.

Chapter 7. CONCLUSION AND FUTURE DIRECTIONS

7.1 Conclusion

This report has executed the complete data science pipeline for the customer churn prediction problem on the IBM Telco Customer Churn dataset: data cleaning (dropping `customerID`, handling 11 hidden missing values in `TotalCharges`), exploratory data analysis, leakage-free data preprocessing, comparison of six classification algorithms, hyperparameter tuning and threshold selection on the training set, and a final evaluation on the test set.

Key conclusions:

1. Churn is strongly associated with short tenure, month-to-month contracts, and electronic check payments (Chapter 4), and the dataset contains sufficient signal for models to achieve an ROC-AUC of approximately 0.84 on the test set.

2. Random Forest with `class_weight = "balanced"`, max_depth $= 10$, min_samples_leaf $=$ 5, n_estimators $= 100$, and a threshold of 0.46 detects 80.7% of churned customers (Recall) with a Precision of 51.9% and an F1 $= 0.632$.

3. Most performance improvements stem from handling class imbalance via class weights rather than increased model complexity; simple Logistic Regression achieves performance comparable to Random Forest in cross-validation.

4. The Pipeline workflow, threshold selection on the training data, and single-test-set evaluation make the reported figures more reliable, though uncertainties due to sample size still warrant consideration.

7.2 Future Directions

- **Model Explainability:** Calculate feature importance (permutation importance, SHAP) to clearly identify which factors contribute most and test the hypotheses stated in Chapter 4.

- **Rigorous Evaluation:** Use repeated or nested cross-validation, and compute bootstrap confidence intervals for test set metrics.

- **Cost-Sensitive Thresholding:** Replace F1 with an expected utility function based on c, q, and v (Section 6.5), combined with probability calibration (Platt scaling, isotonic regression).

- **Model Expansion:** Experiment with XGBoost, LightGBM, CatBoost; test SMOTE or undersampling placed strictly inside the Pipeline to prevent leakage.
- **Survival Analysis:** Directly model the time-to-churn according to tenure rather than binary churn prediction.
- **Customer Lifetime Value (CLV) Integration:** Combine churn probability with expected revenue to prioritize high-value customers.
- **Real-world and Time-series Data:** Validate models on real telecommunications provider data with chronological evaluation.

MEMBER CONTRIBUTIONS

The successful completion of this research project reflects the collaborative efforts, dedication, and shared responsibilities of all group members. The specific contributions and core task allocations are outlined as follows:

1. **Lê Triệu Quang Minh - MSV 11247198** (34%): Spearheaded project conceptualization, structural framework design, data cleaning workflows, missing value imputation strategies, and data leakage prevention protocols (Chapters 3 + 4).
2. **Tạ Quang Huy - MSV 11247175** (33%): Conducted comprehensive exploratory data analysis (EDA), statistical visualization, feature correlation assessment, and documentation of behavioral insights (Chapters 1 + 2).
3. **Vũ Hoàng Quân - MSV 11247220** (33%): Designed and implemented machine learning models, executed GridSearchCV hyperparameter optimization, formulated performance evaluation metrics, and finalized technical documentation (Chapters 5, 6 & 7).

REFERENCES

[1] IBM. (n.d.). *Telco customer churn* [Data set]. GitHub. `https://github.com/IBM/telco-customer-churn-on-icp4d`

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.